

Arduino Cookbook : De la Théorie à la Pratique

Chapitre 17 : Code Avancé et Gestion de la Mémoire

Présenté par : Dr B. DIOP

29 mars 2026

Référence : Michael Margolis, *Arduino Cookbook*, 2nd Edition, O'Reilly

17.0 Introduction : Repousser les limites matérielles

Le Mur de la Mémoire et de la Vitesse

L'Arduino Uno a des ressources extrêmement limitées par rapport à un ordinateur (2 Ko de RAM et un processeur à 16 MHz). Lorsque les projets deviennent complexes (écrans couleurs, web serveurs), le programmeur doit ruser.

Dans ce chapitre, nous allons apprendre à :

- Comprendre l'architecture mémoire (Flash, SRAM, EEPROM).
- Sauver la mémoire vive (SRAM) avec la macro `F()` et `PROGMEM`.
- Stocker des données de manière permanente avec l'EEPROM.
- Accélérer drastiquement les entrées/sorties avec la manipulation de ports.
- Réagir instantanément aux événements avec les Interruptions Matérielles.

17.1 L'Architecture Mémoire de l'Arduino

Un microcontrôleur AVR (comme l'ATmega328P de la Uno) possède **trois espaces de mémoire distincts** :

1. **La Mémoire Flash (32 Ko)** : C'est le "disque dur". Elle stocke votre programme compilé (.hex). Elle n'est pas effacée quand on coupe le courant.
2. **La SRAM (2 Ko)** : C'est la mémoire vive. Elle stocke vos variables dynamiques (int, String, tableaux) pendant que le programme tourne. **C'est le goulot d'étranglement principal.**
3. **L'EEPROM (1 Ko)** : Une petite mémoire non volatile, utile pour stocker des paramètres de configuration à conserver sans électricité.

17.2 Le cauchemar de la SRAM saturée

Que se passe-t-il si vos variables dépassent les 2048 octets (2 Ko) de la SRAM ?

La Collision (Stack vs Heap)

L'Arduino n'affiche **pas** de message d'erreur. Les variables vont simplement s'écraser les unes les autres. Le programme va geler, redémarrer en boucle, ou afficher des caractères aléatoires sur le port série ou l'écran LCD.

Causes communes :

- Utilisation abusive d'objets `String`.
- Trop de texte brut dans les `Serial.print("Une très longue phrase...");` (ce texte est copié en SRAM par défaut!).
- De trop grands tableaux (ex : tampons de réception réseau).

17.3 Mesurer la RAM disponible

Il n'y a pas de fonction native officielle, mais on peut calculer l'espace libre entre le "Tas" (Heap) et la "Pile" (Stack).

```
int freeRam() {  
    extern int __heap_start, *__brkval;  
    int v;  
    return (int) &v - (__brkval == 0 ? (int) &__heap_start : (int) __brkval);  
}  
  
void setup() {  
    Serial.begin(9600);  
    Serial.print("RAM Libre au demarrage : ");  
    Serial.print(freeRam());  
    Serial.println(" octets");  
}  
  
void loop() {}
```

17.4 La Macro Magique : F()

C'est l'astuce la plus facile et la plus efficace pour sauver votre SRAM.

Par défaut, le compilateur copie chaque caractère de `Serial.println("Texte")` dans la SRAM au démarrage. Avec la macro `F()`, on force l'Arduino à lire le texte **directement depuis la mémoire Flash** (qui est 16 fois plus grande!).

```
void setup() {  
  Serial.begin(9600);  
  
  // Mauvaise methode : Consomme 33 octets de SRAM precieuse  
  Serial.println("Bonjour, ceci est un tres long message d'erreur.");  
  
  // Bonne methode : Consomme 0 octet de SRAM !  
  Serial.println(F("Bonjour, ceci est un tres long message d'erreur.));  
}
```

17.5 Stocker des tableaux en Flash : PROGMEM

Si vous avez un tableau de constantes (ex : polices de caractères pour écran OLED, échantillons sonores), utilisez le modificateur PROGMEM.

```
#include <avr/pgmspace.h>

// Place ce grand tableau dans la memoire Flash au lieu de la SRAM
const uint16_t maPolice[5] PROGMEM = { 0xFFFF, 0x0000, 0x1234, 0xABCD, 0x5555 };

void setup() {
    Serial.begin(9600);

    // ATTENTION : On ne peut pas lire maPolice[2] directement !
    // Il faut utiliser une fonction speciale pour aller chercher la donnee en Flash :
    uint16_t donnee = pgm_read_word_near(maPolice + 2);

    Serial.println(donnee, HEX); // Affiche 1234
}
```

17.6 L'EEPROM : La Mémoire de Stockage

Comment sauvegarder le "Meilleur Score" d'un jeu ou l'état d'un relais en cas de coupure de courant ? En utilisant l'EEPROM interne.

```
#include <EEPROM.h>

void setup() {
  Serial.begin(9600);

  // Ecrire dans l'EEPROM (Adresse 0, Valeur 42)
  // Attention : Une cellule EEPROM ne peut stocker qu'un octet (0 a 255) !
  EEPROM.update(0, 42); // update est meilleur que write (economise les cycles)

  // Lire depuis l'EEPROM
  byte valeurSauvee = EEPROM.read(0);
  Serial.print("Valeur recuperee : ");
  Serial.println(valeurSauvee);
}
```

Attention : L'EEPROM a une durée de vie limitée à environ 100 000 écritures par cellule.

17.7 Opérations Bit à Bit (Bitwise)

Les processeurs aiment les zéros et les uns. Souvent, 8 capteurs ON/OFF sont représentés par un seul octet (ex : B10110010).

Les opérateurs logiques de base :

- **AND (&)** : Garde un bit à 1 seulement si les deux sont à 1 (Masquage).
- **OR (|)** : Met un bit à 1 si au moins l'un des deux est à 1 (Forçage).
- **XOR (^)** : Met un bit à 1 si les deux sont différents (Inversion).
- **NOT (~)** : Inverse tous les bits d'un octet.
- **Décalages (<< et >>)** : Pousse les bits vers la gauche ou la droite (équivalent à multiplier ou diviser par 2).

17.8 Fonctions d'aide Arduino pour les bits

L'Arduino fournit des macros faciles à lire pour éviter les erreurs de syntaxe C++ complexes sur les opérations bit à bit.

```
void setup() {  
    byte registre = B10000000; // Un seul bit a 1 (le bit n 7)  
  
    bitSet(registre, 0); // Force le bit 0 a 1 -> B10000001  
    bitClear(registre, 7); // Force le bit 7 a 0 -> B00000001  
    bitWrite(registre, 3, 1); // Ecrit 1 au bit 3 -> B00001001  
  
    // Lire la valeur d'un bit precis  
    if (bitRead(registre, 3) == 1) {  
        // Fait quelque chose  
    }  
}
```

17.9 Le Problème de digitalWrite()

La fonction digitalWrite(broche, HIGH) est facile à utiliser, mais elle est **très lente** (environ 50 à 60 cycles d'horloge).

Pourquoi ? Parce qu'en coulisse, elle désactive le PWM, vérifie si la broche est valide, cherche à quel port matériel elle appartient, puis change l'état.

La solution : La Manipulation Directe des Ports. On peut contourner les fonctions Arduino et écrire directement dans les registres matériels du microcontrôleur AVR pour allumer une LED en **1 seul cycle d'horloge !**

17.10 Les Registres de Ports (DDR, PORT, PIN)

Les broches de l'ATmega328P sont regroupées par blocs de 8, appelés Ports (B, C et D).

Chaque Port est contrôlé par 3 registres (octets) :

- **DDRx (Data Direction Register)** : Équivaut à `pinMode()`. Un bit à 1 = OUTPUT, 0 = INPUT.
- **PORTx (Data Register)** : Équivaut à `digitalWrite()`. Écrire 1 met la broche à 5V, écrire 0 la met à 0V.
- **PINx (Input Pins Register)** : Équivaut à `digitalRead()`. Permet de lire l'état matériel actuel.

Exemple : Le PORTD contrôle les broches numériques de 0 à 7.

Code : Manipulation Directe de Port

Comment faire clignoter la broche 3 (qui appartient au PORTD) sans utiliser les fonctions lentes ?

```
void setup() {  
  // pinMode(3, OUTPUT);  
  // Met le bit 3 du registre DDRD a 1 (Direction Sortie)  
  DDRD = DDRD | B00001000;  
  // Ou en ecriture C++ moderne : DDRD |= (1 << 3);  
}  
  
void loop() {  
  // digitalWrite(3, HIGH);  
  PORTD |= (1 << 3); // Force le bit 3 a 1 sans toucher aux autres (OR)  
  
  delay(1000);  
  
  // digitalWrite(3, LOW);  
  PORTD &= ~(1 << 3); // Force le bit 3 a 0 sans toucher aux autres (AND + NOT)  
  
  delay(1000);  
}
```

17.11 Les Interruptions Matérielles (Hardware Interrupts)

L'un des concepts les plus puissants en électronique. Au lieu de vérifier en boucle si un bouton est pressé (*Polling*), on demande au processeur de nous prévenir de lui-même.

Le principe : Dès qu'un signal (ex : front montant) arrive sur une broche d'interruption, le processeur met en pause immédiate le programme `loop()`, saute exécuter une petite fonction spéciale (ISR), puis reprend le `loop()` là où il s'était arrêté.

Sur l'Arduino Uno, seules les broches 2 et 3 supportent les interruptions matérielles externes.

Code : Implémenter une Interruption

```
const byte boutonPin = 2;
// 'volatile' est OBLIGATOIRE pour une variable modifiée dans une ISR
volatile bool boutonPresse = false;

void setup() {
  Serial.begin(9600);
  pinMode(boutonPin, INPUT_PULLUP);
  // Attache l'interruption à la broche 2, déclenchée sur un front descendant
  attachInterrupt(digitalPinToInterrupt(boutonPin), declenchementISR, FALLING);
}

void loop() {
  if (boutonPresse) {
    Serial.println("Le bouton a été pressé instantanément !");
    boutonPresse = false; // Remet le drapeau à zéro
  }
}

// L'ISR (Interrupt Service Routine). Doit être la plus COURTE possible !
// Interdit d'utiliser delay() ou Serial.print() ici !
void declenchementISR() {
  boutonPresse = true;
}
```

Résumé du Chapitre 17

- **RAM** : Estimez la mémoire utilisée. Un `Serial.println(F("..."))`; systématique sauvera vos plus gros projets de la collision mémoire.
- **Types de données** : N'utilisez un `int` (2 octets) que si la valeur dépasse 255. Sinon, utilisez un `byte` ou `uint8_t` (1 octet).
- **Vitesse** : La manipulation directe de port (`PORTB |= B00100000;`) est dangereuse mais indispensable pour générer des signaux à très haute fréquence (VGA, DMX).
- **Interruptions (ISR)** : Elles garantissent qu'aucun événement externe court (codeur rotatif, signal radio) ne sera ignoré par un `delay()` dans votre boucle principale.

Fin du Chapitre 17. Souhaitez-vous terminer avec le Chapitre 18 consacré à l'utilisation de contrôleurs et puces matérielles (Using Controller Chips) ?